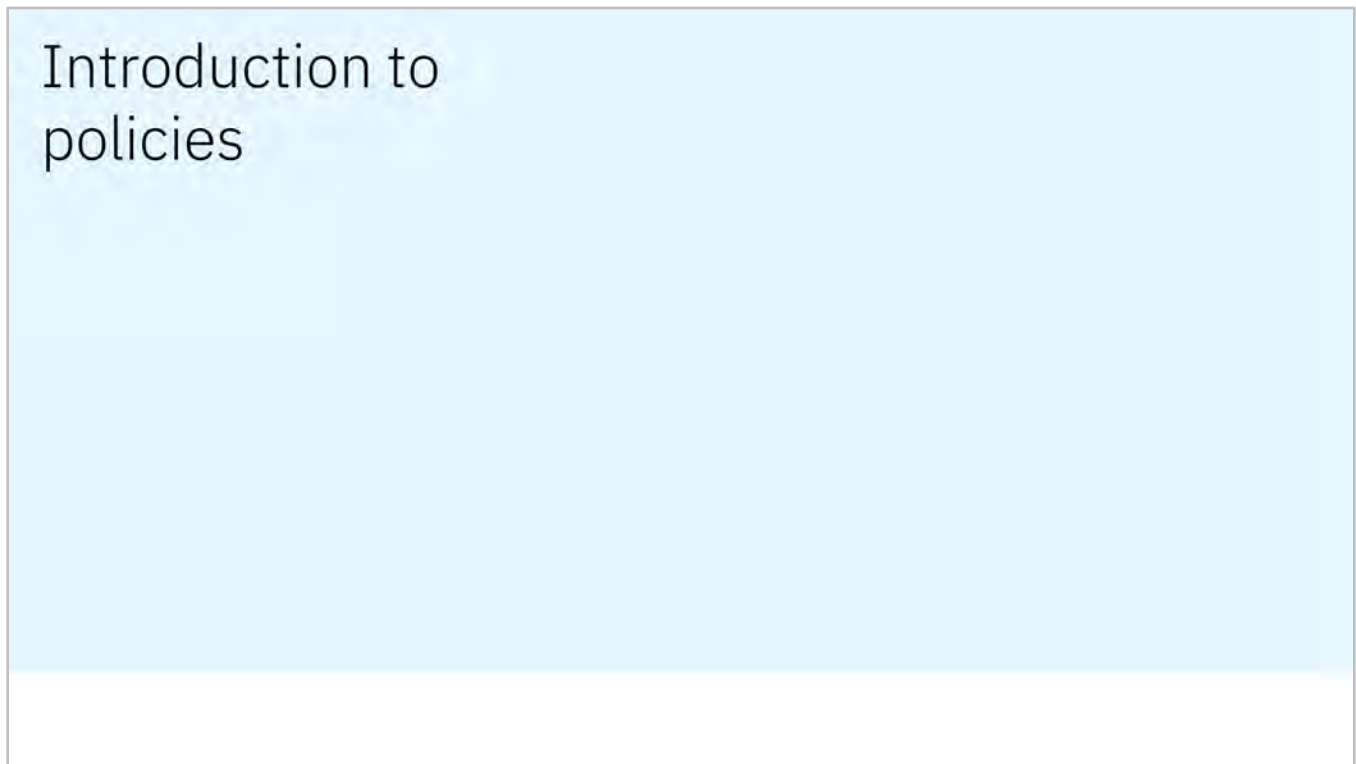


Introduction to policies

Figure: Introduction to policies



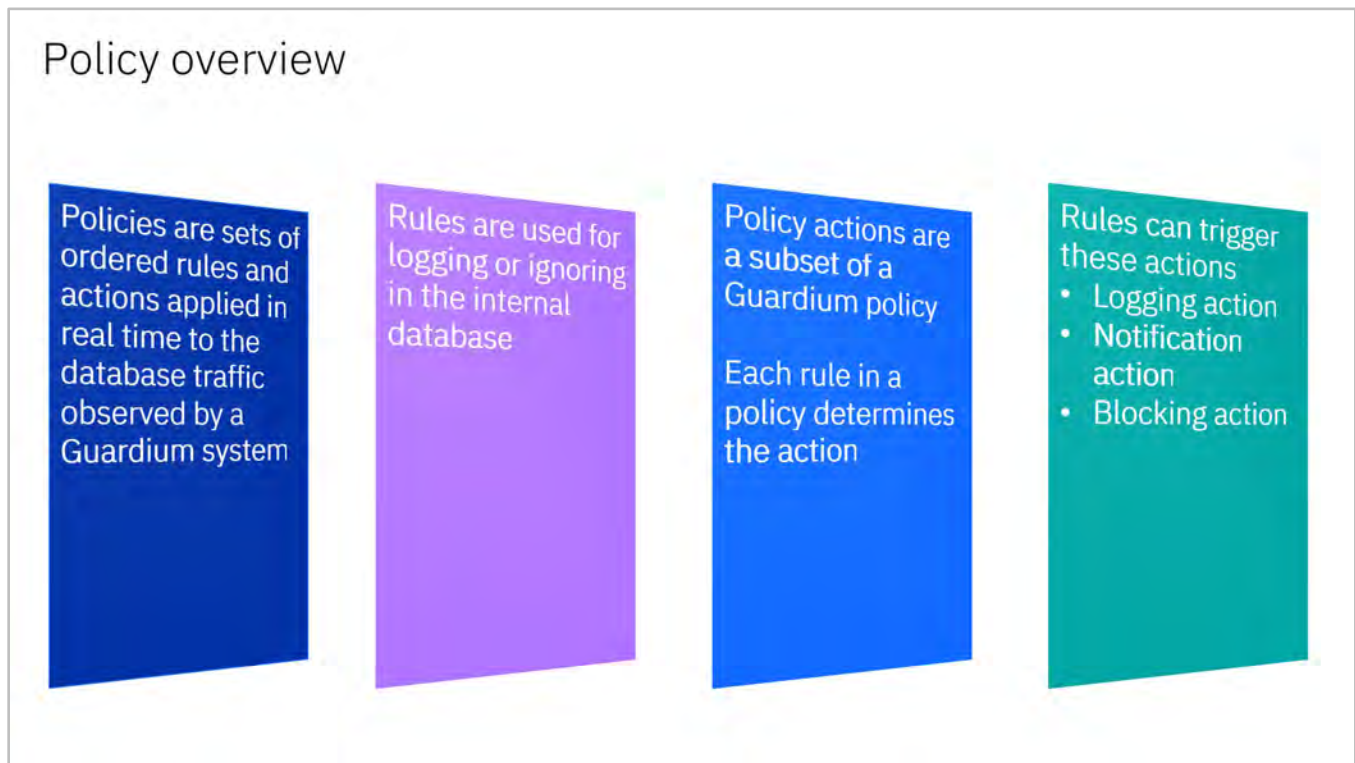
A policy is a set of rules that returns data about your business. Before writing rules, understand how Guardium collects information.

In a typical deployment environment, databases might process millions of database requests each hour. That means that the S-TAP agents send much information to the Guardium collectors, which in turn must parse and store that information.

Efficient policy rules can reduce the amount of data to process and store. For example, you might want to ignore certain database queries or even entire sessions. Other queries might need an immediate response.

This lesson introduces Guardium policies and rules.

Figure: Policy overview



Effective data security requires you to create, maintain, and implement policies.

Policies are sets of ordered rules and actions that are applied in real time to the database traffic that a Guardium system observes.

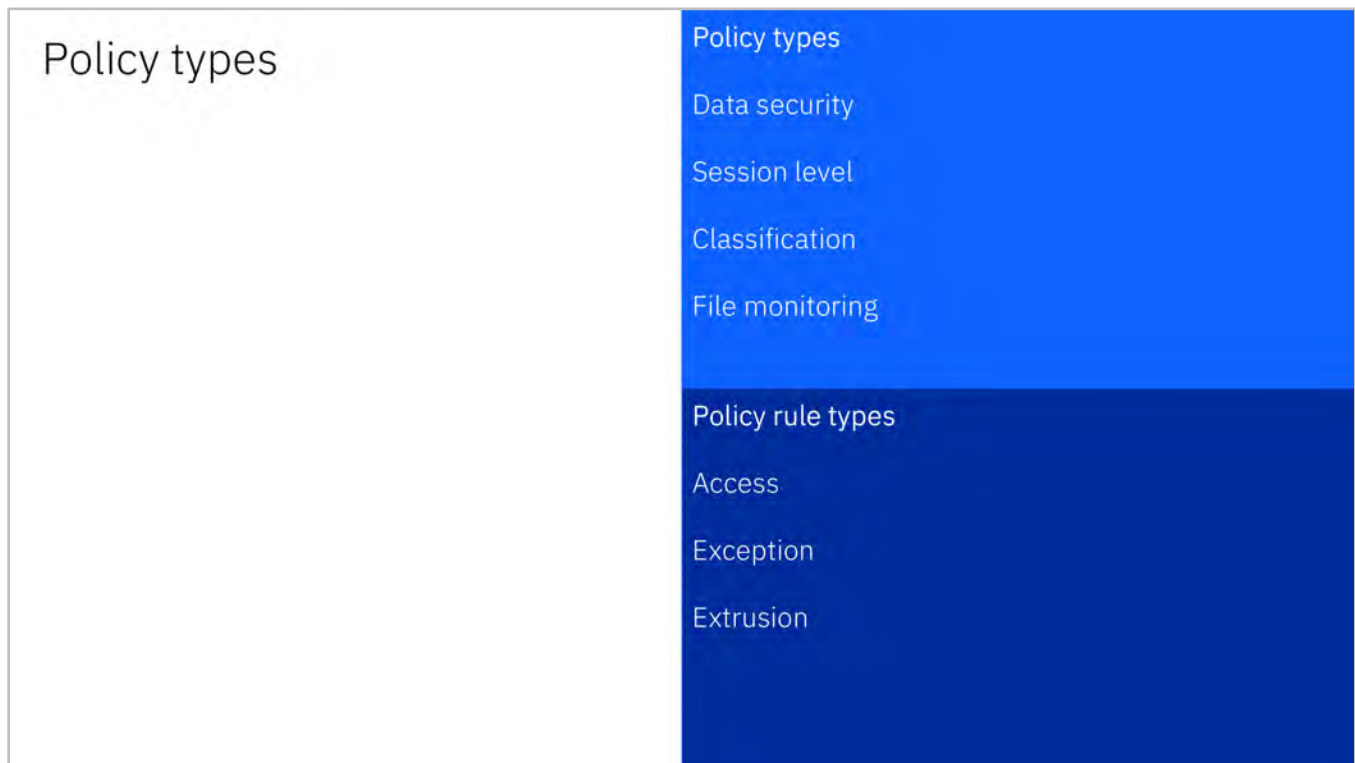
Rules are used to log or ignore traffic. Each rule can apply to a request from a client, or to a response from a server. Within a policy, rules are evaluated in the order in which they appear, as each element of traffic is analyzed.

Policy actions are a subset of a Guardium policy. Each rule in a policy defines a conditional action that tells the sniffer on the collector how to process the events.

The condition that is tested can be a simple test, for example, it might check for any access from a client IP address that does not belong to an authorized client IPs group. Or the condition that is tested can be a complex test that considers multiple message and session attributes (database user, source program, command type, time of day, and so on) and is sensitive to the number of times the condition is met within a specified time frame. The rule itself determines the action.

Policy actions define which traffic is **ignored** or **logged**, or which activities trigger an **alert** or **block** access to the database. You can define and install multiple policies on a Guardium system at the same time.

Figure: Policy types



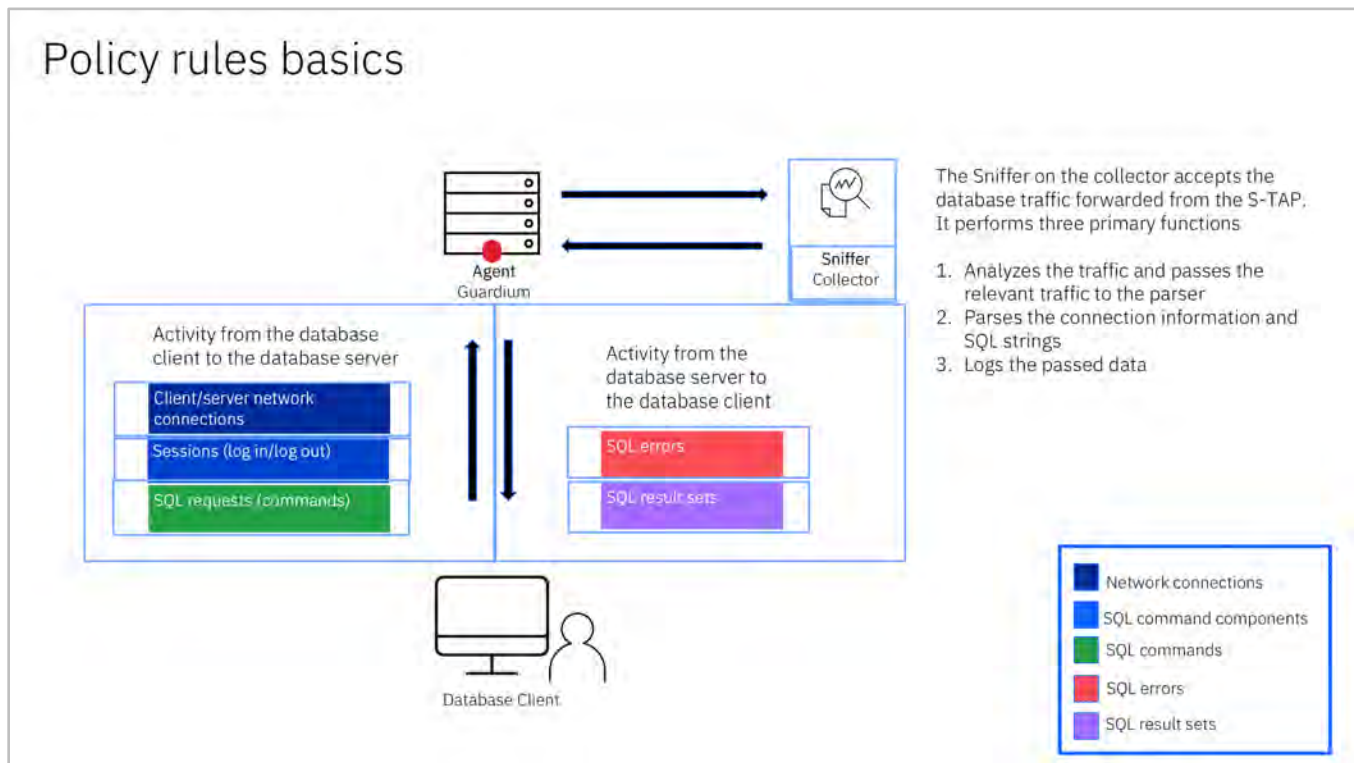
Policies are present on the central manager but sync to and work on the collector.

- Guardium provides several types of policies:
- Data security policies contain rules to apply to the observed traffic between database clients and servers. This policy type is the most versatile policy.
- Session level policies are like data security policies but improve performance by validating criteria at the beginning of sniffer processing.
- Classification policies and processes define how Guardium discovers and treats sensitive data such as credit card numbers, social security numbers, and personal financial data.
- A file monitoring policy specifies how Guardium handles different file activity events.

Policies contain three types of policy rules:

- An access rule applies to client requests.
- An exception rule evaluates exceptions that the server returns.
- An extrusion rule evaluates data that the server returns in response to a request.

Figure: Policy rules basics



After the inspection engines are configured, S-TAP forwards all database traffic to the collector. When the sniffer receives the traffic from S-TAP, it performs three functions against the data:

1. **Analyzes** the data to verify that it is valid SQL traffic
2. **Parses** the data for reporting
3. **Logs** the parsed data to the internal database in Guardium

Policies can be created on any appliance, but they apply to the entire environment. In general, a policy is created on the central manager and then pushed to MUs (collector appliances).

The diagram on this slide also shows how server and database client traffic map to the three different types of rules.

An **access** rule applies to client requests. It captures traffic that goes from the database client to the database server. For example, it might test for update commands that are issued from a specific group of IP addresses. The client itself might be an application or a user, for example. Access rules concern themselves with the following different types of traffic, which flows between the database client and the database server. Data passes through these layers in a particular order when information is sent:

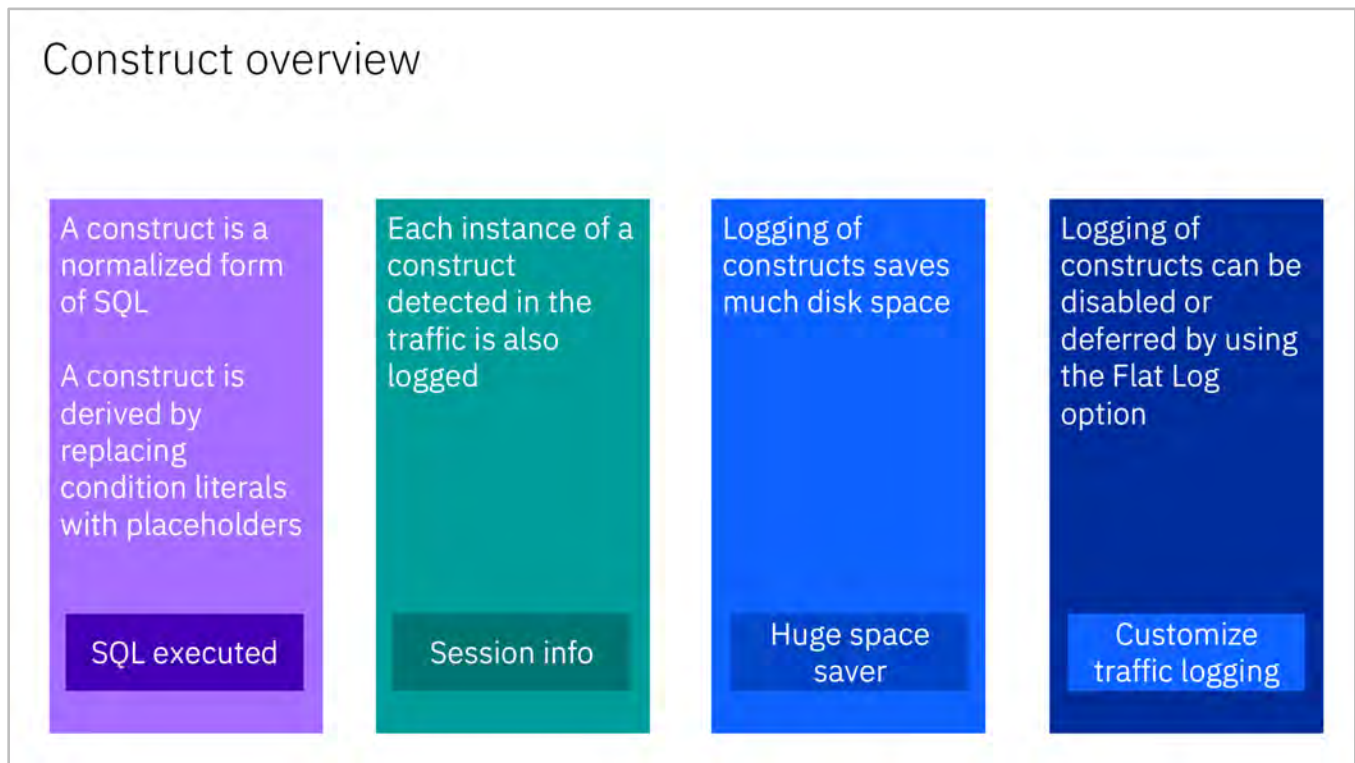
- At the base, there is a client/server network connection.
- The network connection might have one or more network sessions connected.
- Each session can have multiple SQL requests associated with that session.

Exception and **extrusion** rules concern themselves with SQL results and SQL errors in the traffic that passes from the database server to the database client.

- An **exception** rule is concerned with SQL errors that are thrown by the SQL server. For example, it might test for five login failures within one minute.
- An **extrusion** rule is concerned with SQL results that the database returns to the client. For example, it might test the returned data for numeric patterns that might be social security or credit card numbers.

If a rule triggers, results are usually logged as a policy violation. To view policy violations logged in Guardium, go to **Comply > Reports > Incident Management**. There, you can view the Policy Violations and Incident Management report.

Figure: Construct overview



What are constructs and how do they work? Guardium uses constructs to efficiently log data while saving disk space.

- A construct is a normalized form of SQL.
- Constructs are prototypes of requests that Guardium detects in traffic. The combinations of commands, objects, and fields that are included in a construct can be complex, but each construct represents a specific access request type. The detection and logging of new constructs begins when the inspection engine starts, and by default, with one exception, it continues regardless of any security policy rules.
- A construct is derived by replacing condition literals with placeholders. It is an SQL statement text string from a list of data items and literals.
- Usually, the exact SQL is not important, just the type of SQL.
- Each construct instance that is detected in the traffic is also logged, and each instance is related to a specific client/server session. No SQL is stored for a construct instance, except when a policy rule requests the logging of SQL for that instance, or for a particular client/server session of instances (with or without values).

Note: GDM_CONSTRUCT_INSTANCE table: This specific table links both the session information (GDM_SESSION) and the SQL that runs in each session.

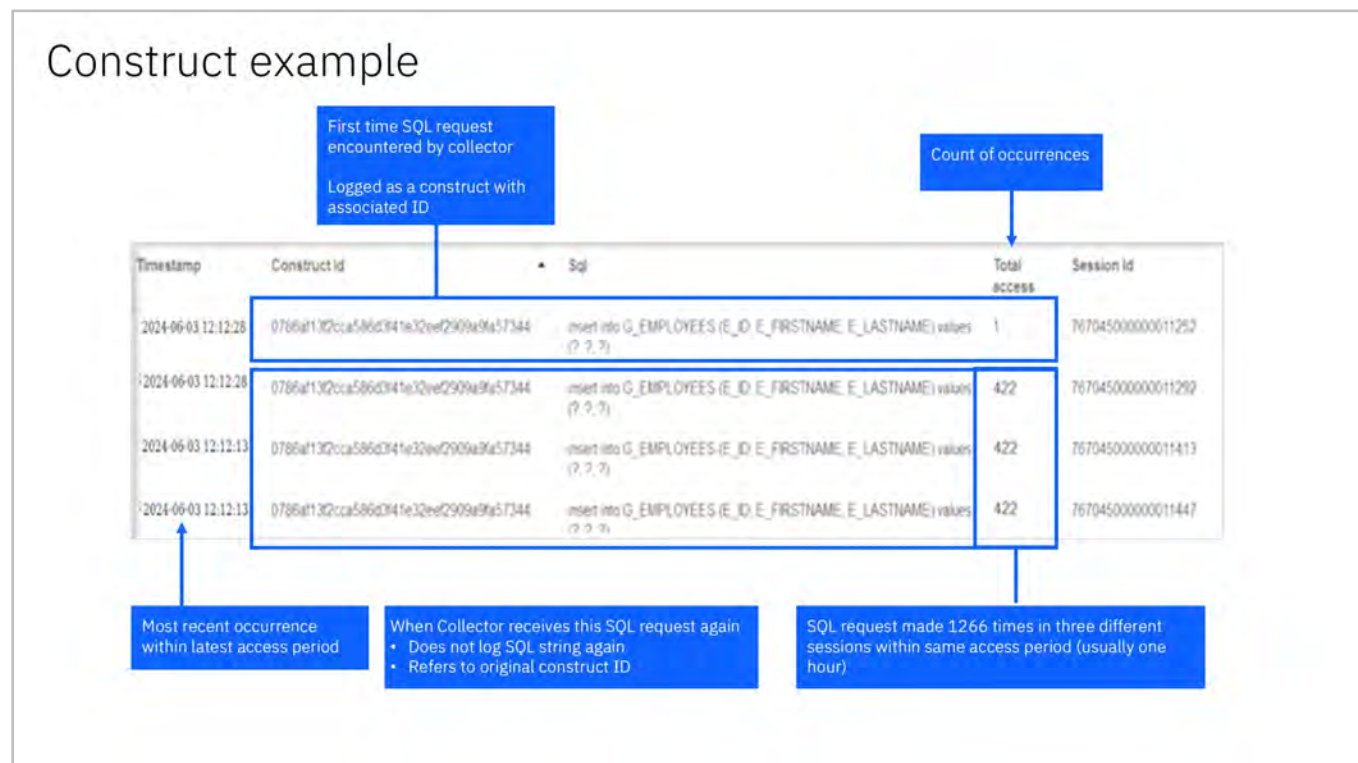
By default, Guardium logs SQL constructs and related SQL instances in the GDM_CONSTRUCT_INSTANCE table. If a session has duplicate SQLs, after the SQL construct and its related instance record are logged, the sniffer counts the number of times it receives the construct. The sniffer updates the Access Period Timestamp to the time of the

last request in the database. Duplicate requests within that session result in an increase in the “Count” column for that SQL instance. Thousands of requests can be collapsed into just a few lines. If each line is written separately, the disk fills up quickly. In a production environment, millions of lines per hour can be saved in this manner. This logging method prevents excessive logging and saves much space.

In addition to controlling the inclusion of SQL in client construct instances, a security policy rule can disable the logging of constructs and instances for the remainder of a session.

In heavy volume situations, the parsing and aggregating of information into constructs and instances can be deferred by using the Log Flat (Flat Log) option. When used, the production of alerts and reports are delayed until the logged information is aggregated.

Figure: Construct example



When the sniffer encounters an SQL request that it did not encounter before, it logs the request as a construct with an associated primary key. The sniffer logs the sentence by replacing the actual values that the user entered with question marks. This replacement is done for two reasons by using constructs:

- These values can be sensitive; therefore, it is not good for Guardium to log this information automatically, which might risk displaying the information to unauthorized users.
- By masking the values, Guardium can increase the data retention on the collectors and aggregators.

In the example on the slide, the sniffer logged three entries.

If the sniffer receives the same construct multiple times within the defined access period (usually one hour) and within the same session, it counts the number of times it receives the construct and updates the access period timestamp to the time of the last request. Therefore, in reporting the finest level of detail, you see how many times the construct ran within an hour, with a timestamp that represents the most recent occurrence. Without constructs, there would be 1266 lines that were logged.

When the sniffer receives the same construct multiple times over an extended period, it makes new entries in the database in two circumstances:

- The user starts a new session.

When a new session starts, a new record is entered with its own access period timestamp and counter. All further occurrences of this construct within this session update this record's access period timestamp and counter until a new access period begins.

- A new access period begins within the same session.

The default access period is one hour (9:00–9:59, 10:00–10:59, and so on.). When a new access period begins, the next occurrence is entered as a new line with its own access period timestamp and counter.

Example: Constructs are logged with values that are replaced with question marks, which make most SQL requests less unique. For example, the following statements appear to be unique to each other:

```
select * from employee_table where employee_id = 48 and hire_date = '8/2/24'
```

```
select * from employee_table where employee_id = 4940 and hire_date = '10/29/24'
```

However, if replacing the values with questions marks, see that they are the same basic request:

```
select * from employee_table where employee_id = ? and hire_date = ?
```

```
select * from employee_table where employee_id = ? and hire_date = ?
```

The string `select * from employee_table where employee_id = ? and hire_date = ?` is an example of a construct. When the sniffer first encounters this SQL request, it logs the request with an associated construct ID. When the sniffer encounters it again, it does not log the request a second time. Instead, the sniffer refers to the construct it logged earlier.

